1 Approach

The hardware is a fully synchronous, streaming pipeline processing one pixel per clock cycle. The architecture is broken into 5 main stages:

- **Pixel Stream Input:** Accepts a continuous 8-bit grayscale stream with `valid_in` handshaking.
- **Gaussian Blur Pre-processing:** Implements a 3x3 Gaussian kernel before edge detection to suppress high-frequency noise.
  - Uses a multi-stage adder tree and a bit-slice (`[11:4]`) for a hardware-efficient divide-by-16 normalization.
- **Line Buffering (Memory):** Uses two cascaded line buffers per module.
  - **Design Choice: BRAM** instead of Distributed RAM. This utilizes dedicated FPGA SRAM, saving hundreds of logic slices while providing a 1-clock latency FIFO. (Discussed in detail in the design tradeoffs section).
- **3x3 Sliding Window:** Extracts the spatial neighborhood using a grid of 9 flip-flops.
  - Includes explicit delay registers (`p_in_d2`, `lb1_out_d1`) to compensate for BRAM read latency, ensuring perfectly aligned vertical columns.
- **Sobel Computation & Magnitude Approximation:** Computes G_x and G_y using a pipelined adder-subtractor tree, expanding bit-widths to 12-bit signed integers to prevent overflow.
  - **Design Choice:** Replaces the expensive square root operation with the L1 Norm approximation ($|G| \approx |G_x| + |G_y|$). This prevents floating points achieving single-cycle latency and massive area savings.
- **Output & Thresholding:** Compares the final magnitude against a configurable 8-bit threshold, outputting 8'hFF (edge) or 8'h00 (background).

2 Block Diagram

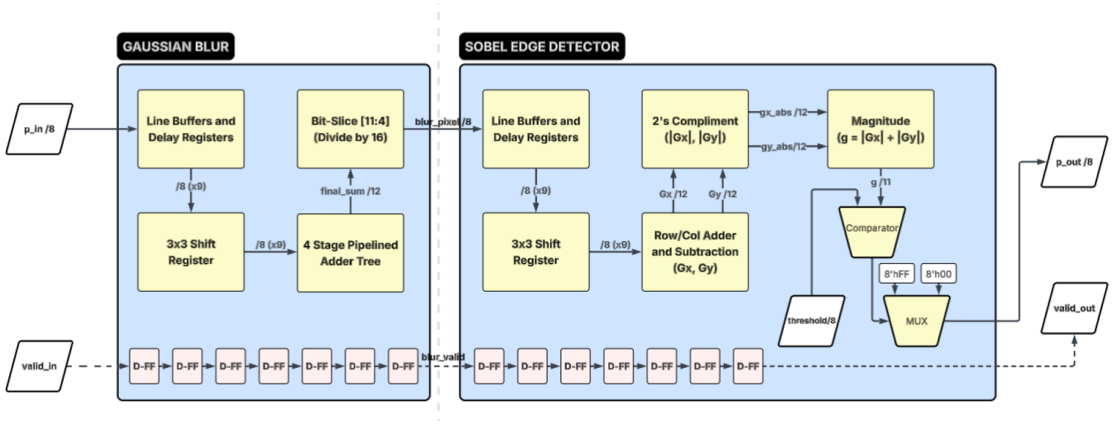


Figure 1: Hardware Architecture Block Diagram

3 Pipeline Latency

3.1 Computational Latency (L_c)

The number of clock cycles required for a single pixel to propagate through the internal pipeline registers from input to output, independent of image dimensions.

$L_c = D_{Gaussian} + D_{Sobel}$

- $D_{Gaussian}$: 3 clocks (window alignment delays) + 5 clocks (adder tree pipeline) = 8 clocks.
- D_{Sobel}: 3 clocks (window alignment delays) + 5 clocks (math pipeline) = 8 clocks.
- L_c: 16 clock cycles.

The very first output pixel which is the black border padding emerges 16 clock cycles after the first input pixel.

3.2 Spatial Latency (L_s)

$L_s = 4W + 20$ (W is the width of the image)

- **Gaussian Buffer Fill:** It takes 2W clocks for the incoming pixel stream to push data through lb1 and lb2 so that three distinct rows are stacked vertically.
- **Sobel Buffer Fill:** Once Gaussian starts streaming, Sobel requires another 2(W-2) clocks to fill its own 3-row stack.
- **Alignment & Stalls:** +8 clocks (2 for Gaussian horizontal alignment, 2 for Sobel horizontal alignment, and 4 idle cycles as Sobel waits through blanked row boundaries).
- **Total Spatial Latency Formula:** $2W + 2(W - 2) + L_c + 8$

For Width (W) = 128:

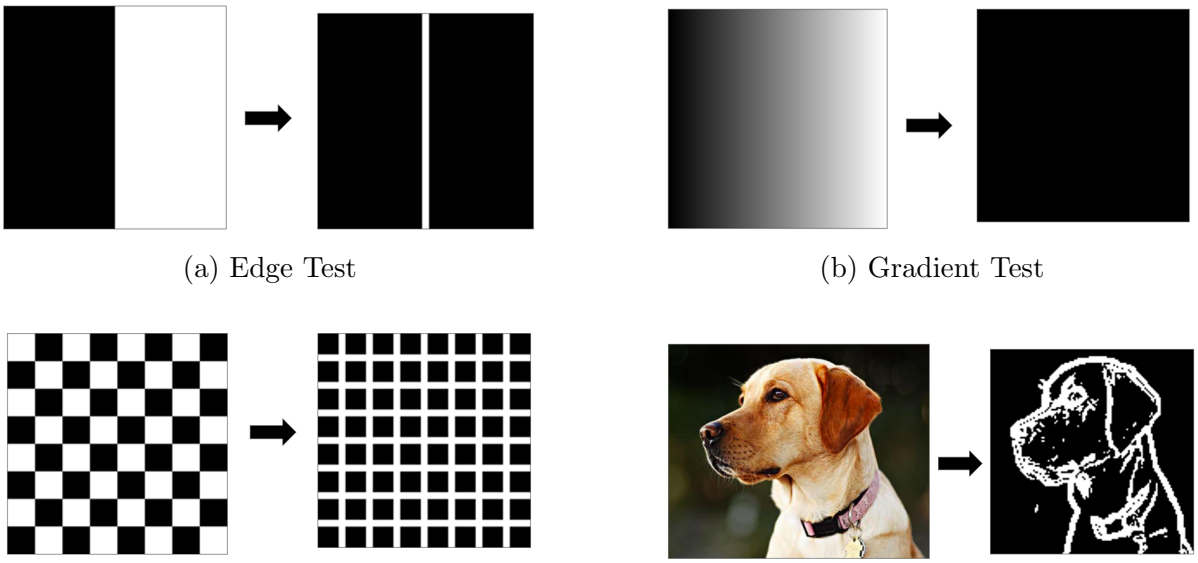
- Computational Latency: **16 clock cycles**
- Spatial Latency: $2(128) + 2(126) + 16 + 8 = 256 + 252 + 24 =$ **532 clock cycles**

4 Border Handling

The design uses a cropping strategy with no padding. As the 3x3 window passes over the edges, invalid boundary pixels are by default dropped.

- The input image is **128x128**.
- The Gaussian Blur stage consumes 2 rows and 2 columns for its window, outputting a **126x126** valid stream.
- The Sobel module is parameterized to WIDTH = 126 to perfectly catch this shrunken stream. It consumes another 2 rows and columns, resulting in a final valid output of **124x124** pixels.
- This approach avoids complex border-multiplexing logic, saving significant FPGA area with giving the output a size of (W-4 x W-4).

5 Outputs



(a) Edge Test

(b) Gradient Test

(c) Checkerboard Test

(d) Real-World Validation

Figure 2: Output Test Results

1. **Edge Test:** The hardware perfectly detects the sudden 00 to FF pixel transition. The pipeline calculates a massive spike in the G_x horizontal gradient, producing a sharp, precisely aligned vertical line with no spatial bleeding.

2. **Gradient Test:** The gradual horizontal fade results in a constant, low-level gradient magnitude. Because this slow rate of change never exceeds the configured threshold, the hardware correctly outputs a solid black image, proving it ignores smooth surfaces.

3. **Checkerboard Test:** The hardware successfully processes simultaneous horizontal (G_x) and vertical (G_y) gradients at maximum capacity without dropping valid signals or corrupting the arithmetic pipeline, producing a perfect grid.

4. **Real-World Validation:** An arbitrary photograph (converted to 128x128 and greyscale via a python script) validates the complete Canny-like front end. The Gaussian pre-processing stage successfully smooths out micro-textures (like fur noise) before the Sobel stage computes the gradients, resulting in a clean outline of the macroscopic subject.

6 Waveform Simulation

6.1 Pipeline Latency

This waveform proves the hardware spatial latency. The `valid_out` signal asserts exactly 532 clock cycles (5.32 microseconds) (removing the additional waits from the testbench made) after the first input, confirming the four line buffers correctly cache the initial image rows before math operations begin.

6.2 Edge Response

This captures the pixel stream crossing the black-to-white boundary.

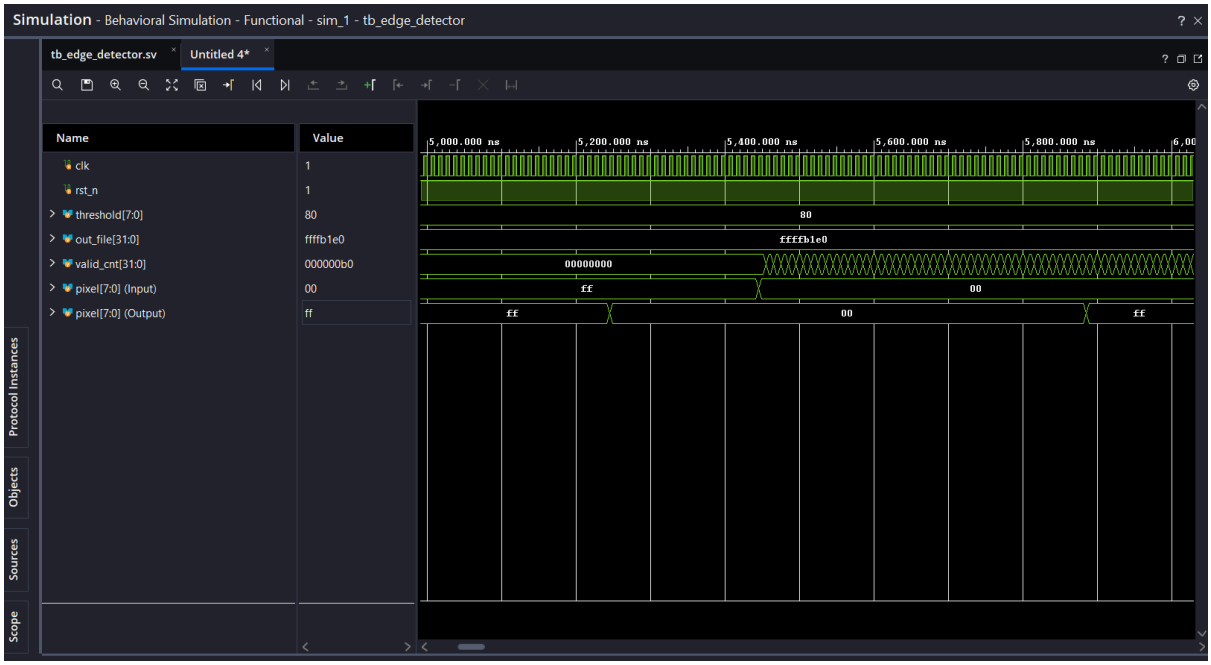


Figure 3: Simulation Waveform for Latency and Edge Response

7 Resource Estimates

- **LUT (Look-Up Tables):** 237
- **FF (Flip-Flops / Registers):** 449
- **BRAM (Block RAM):** 2
- **DSP (Digital Signal Processors):** 0

The RTL was synthesized using Xilinx Vivado, yielding the following resource utilization:

- **Slice LUTs:** LUT utilization is low. The logic is limited to the adder trees, 2's complement inversion, and magnitude comparator.
- **Slice Registers:** The relatively high flip-flop count is a direct result of the deep pipelining architecture. This includes the 18 dedicated sliding window registers (9 per module), the pipeline arithmetic stage registers, and the parallel control-path registers (v1-v7).
- **Block RAM Tiles (2):** The design requires 4 distinct line buffers (two per module), each holding 128 bytes. Because a standard FPGA BRAM tile (typically 18Kb or 36Kb) is much larger than 128 bytes, the Vivado synthesis tool efficiently packed the four logical line buffers into just 2 physical BRAM tiles, proving the memory architecture is highly area-efficient.
- **DSP Slices (0):** This validates the architectural decision to replace the standard Sobel square root magnitude with the L1 Norm approximation, completely freeing the FPGA's dedicated multiplier blocks for other potential system tasks.

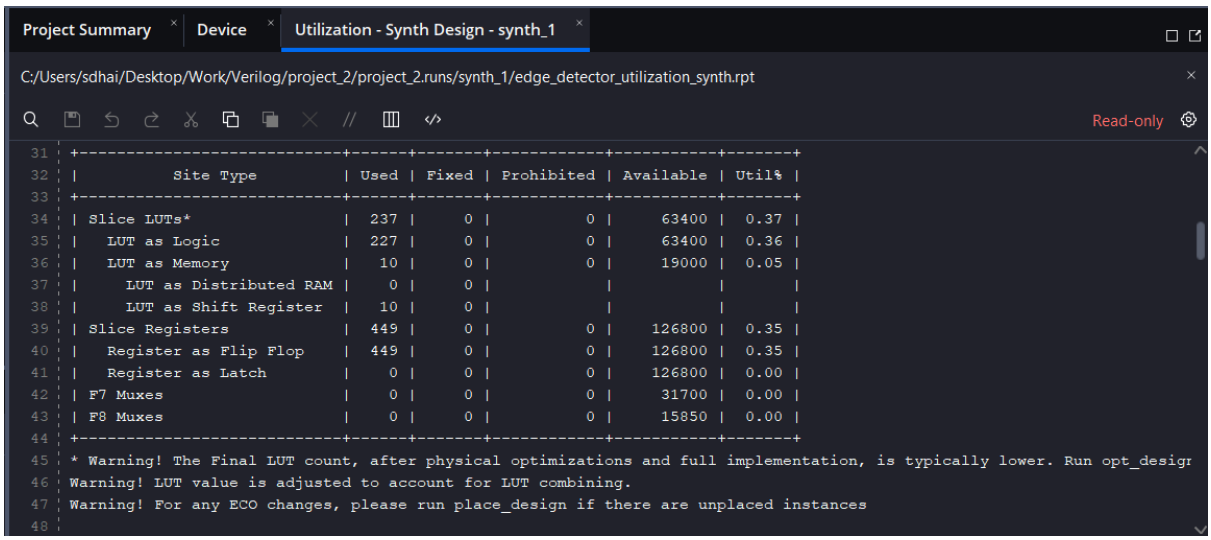


Figure 4: Design Report

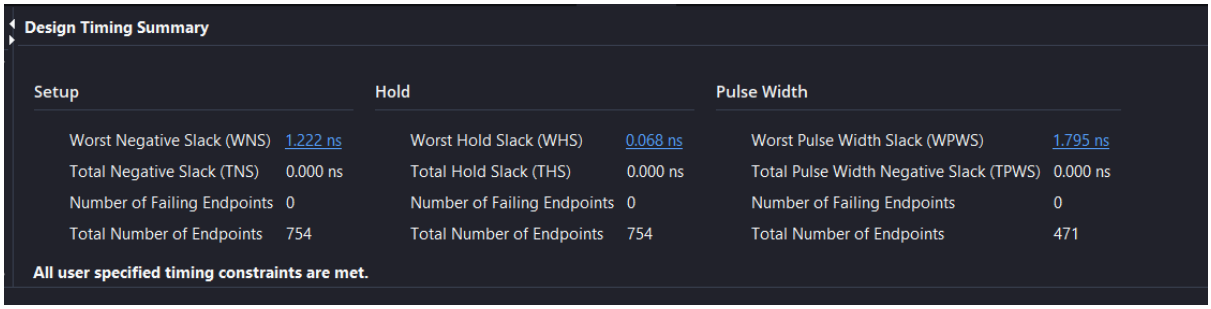


Figure 5: WNS

8 Design Tradeoffs

The following key trade-offs were made:

8.1 Line Buffer Memory: BRAM vs. Distributed RAM

- **Trade-off:** Area vs. Latency.
- **Justification:**
  - Distributed RAM uses the FPGA's general-purpose logic slices (LUTs), which would consume hundreds of logic cells just to delay pixels, limiting the space available for arithmetic routing.
  - By designing the line buffers with explicit read/write pointers, the synthesis tool was forced to infer Block RAM (BRAM).
  - While BRAM introduces a mandatory 1-clock read latency, which required additional alignment registers, it leverages dedicated on-chip SRAM, keeping LUT utilization extremely low and power consumption optimized.

8.2 Border Handling: No-Padding vs. Zero-Padding

- **Trade-off:** Logic Complexity vs. Image Resolution.
- **Justification:**
  - Implementing a zero-padded border requires dynamic coordinate tracking and heavy multiplexing to inject 8'h00 when the 3x3 window crosses an edge.
  - This bloats the critical path and lowers the maximum operating frequency. Instead, no padding strategy was chosen.
  - The pipeline shrinks the 128x128 input into a 124x124 output as it passes through the Gaussian and Sobel windows. Removing a 2-pixel boundary drastically reduced multiplexer logic and prioritized maximum hardware throughput.

8.3 Filter Architecture: Cascaded 3x3 vs. Monolithic 5x5 Kernel

- **Trade-off:** Pipeline Depth vs. Combinational Delay.
- **Justification:** Mathematically, passing an image through a 3x3 Gaussian and then a 3x3 Sobel is identical to passing it through a single, pre-computed 5x5 convolution matrix.
  - Both approaches require exactly four line buffers to cache the spatial data.
  - However, a single 5x5 filter requires an enormous combinational adder tree to sum 25 pixels in a single stage, which creates a massive propagation delay and ruins the clock speed.
  - By cascading two separate 3x3 modules, the math is split. The pipeline depth increases (higher clock latency to first pixel), but the combinational logic between registers is kept short, allowing the FPGA to achieve a much higher maximum clock frequency.

8.4 Magnitude Approximation: L1 Norm vs. True Magnitude

- **Trade-off:** Area vs. Precision.
- **Justification:** The true Sobel magnitude requires squaring the gradients and calculating a square root.
  - This would require DSP slices (multipliers) and complex multi-cycle logic. By using the Manhattan distance approximation ($|G| = |Gx| + |Gy|$), the design completely eliminates DSP usage, replacing it with a single lightweight adder.